7. Automatic Reference Counting and Memory Management

Task 1:

Code:

```
import Foundation
class Person {
    var name: String
    var apartment: Apartment?
    init(name: String) {
        self.name = name
    }
    deinit {
        print("Person \(name) deallocated")
    }
}
class Apartment {
    var unit: String
    weak var tenant: Person?
    init(unit: String) {
        self.unit = unit
    }
    deinit {
        print("Apartment \(unit) deallocated")
    }
}
var person: Person? = Person(name: "Brinda")
var apartment: Apartment? = Apartment(unit: "A101")
person?.apartment = apartment
```

apartment?.tenant = person

person = nil

apartment = nil

Output:

```
Output

Person Brinda deallocated
Apartment A101 deallocated
```

Task 2:

Swift Code – Strong Capture (Retain Cycle):

```
class Downloader {
    var name: String
    var onFinish: (() -> Void)?
    init(name: String) {
        self.name = name
    }
    func start() {
        onFinish = {
            print("Finished \(self.name)")
        }
    }
    deinit {
        print("Downloader deallocated")
    }
}
```

```
var downloader: Downloader? = Downloader(name: "File1")
downloader?.start()
downloader?.onFinish?()
downloader = nil
```

Output:

```
Output

Finished File1
```

Corrected Swift Code – Weak Capture

```
class Downloader {
    var name: String
    var onFinish: (() -> Void)?
    init(name: String) {
        self.name = name
    }
    func start() {
        onFinish = { [weak self] in
            guard let self = self else { return }
            print("Finished \(self.name)")
        }
    }
    deinit {
        print("Downloader deallocated")
    }
}
var downloader: Downloader? = Downloader(name: "File1")
```

```
downloader?.start()

downloader?.onFinish?()

downloader = nil
```

Output:

```
Output

Finished File1
Downloader deallocated
```

Task 3:

Code:

```
class Box {
   deinit {
      print("Box gone")
   }
}
var a: Box? = Box()
var b = a
var c = a
a = nil
b = nil
print("before c")
c = nil
print("end")
```

Output:

```
Output

before c
Box gone
end
```

Table:

Reference	When to use	Can be nil?
Strong	When the reference should keep the object alive.	The reference itself does not automatically become nil.
Weak	When the referenced object can be deallocated independently, such as a delegate.	Yes
Unowned	When the referenced object is guaranteed to outlive the reference.	No